Lab 4 Network Traffic Analysis Using Scapy

Checkpoint 1

- **How many TCP, TCP-port-80, and UDP-port-53 packets did you find?**

```
tcp_packets = []

for pkt in packets:
    if TCP in pkt:
        tcp_packets.append(pkt)

print("TCP packets:", len(tcp_packets))

http_packets = []

for pkt in tcp_packets:
    if pkt[TCP].sport == 80 or pkt[TCP].dport == 80:
        http_packets.append(pkt)

print("TCP port 80 packets:", len(http_packets))

dns_packets = []

for pkt in packets:
    if UDP in pkt:
        if pkt[UDP].sport == 53 or pkt[UDP].dport == 53:
            dns_packets.append(pkt)

print("UDP port 53 packets:", len(dns_packets))
```

```
TCP packets: 474035
TCP port 80 packets: 22430
UDP port 53 packets: 57
```

TCP: 474035

TCP-port-80: 22430

UDP-port-53: 57

- **Why might a filter return zero packets?**

  A filter might return zero packets due to overly strict or incorrect criteria, the absence of that specific traffic in the PCAP file, or attempting to access a non-existent protocol layer.

- **What is the difference between filtering by protocol and filtering by port?**

Protocol filtering selects packets based on the overarching network or transport protocol, while port filtering targets specific communication endpoints within that protocol to identify particular services (such as port 80 for HTTP).

Checkpoint 2

```
packets[0].show()
```

```
###[ Ethernet ]###
  dst       = 08:00:27:b5:b7:19
  src       = 00:1e:49:db:19:c3
  type      = ARP
###[ ARP ]###
     hwtype    = Ethernet (10Mb)
     ptype     = IPv4
     hwlen     = 6
     plen      = 4
     op        = who-has
     hwsrc     = 00:1e:49:db:19:c3
     psrc      = 147.32.84.1
     hwdst     = 08:00:27:b5:b7:19
     pdst      = 147.32.84.165
###[ Padding ]###
        load      = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
```

- **What do source and destination IP addresses tell you?**

  They identify the sender and receiver of the packet, showing the path and direction of network communication.

- **What is the purpose of TCP flags?**

  They control and describe the state of a TCP connection, such as establishing, maintaining, or terminating a session.

- **Why are timestamps useful when investigating possible flooding?**

  They allow analysts to measure the arrival rate of packets and detect abnormal spikes or high-frequency traffic over time.

Checkpoint 3

- **Did you find readable payload text?**

```
for pkt in packets:
        if Raw in pkt:
                sample = bytes(pkt[Raw].load)[:80]
                print(sample.decode("utf-8", errors="replace"))
                break
```

```
PING :barjavel.freenode.net
```

Yes, readable plaintext (such as the PING command) was successfully extracted and decoded from the payload bytes.

- **What might plaintext HTTP expose?**

  It might expose sensitive information such as usernames, passwords, cookies, session tokens, and website contents.

- **Why can you not inspect the contents of encrypted HTTPS traffic using these simple commands?**

  Because HTTPS traffic is encrypted using TLS/SSL, the payload consists of randomized ciphertext that cannot be read using simple decoding commands without decryption keys.

Reflection questions

1. **Why is packet inspection important for network security?**

   It allows deep analysis of network traffic content and behavior to detect malicious attacks, intrusions, anomalies, and policy violations.

2. **What types of security risks can arise from unencrypted traffic?**

   Eavesdropping, Man-in-the-Middle (MitM) attacks, and the leakage of sensitive data (such as passwords, credentials, and plain text).

3. **What are the limitations of passive packet sniffing and PCAP analysis?**

   It is post-event (cannot block attacks in real-time), ineffective against encrypted traffic (TLS/HTTPS), and resource-intensive for high-volume traffic.

4. **How does encryption help prevent information leakage?**

   It transforms plain text into ciphertext, ensuring that unauthorized parties cannot read the intercepted data.

5. **Why might a simple packet-rate detector produce false positives?**

   Legitimate high-volume activities (such as video streaming, online gaming, large downloads, or software updates) can generate high packet rates and trigger false alarms.

Screenshots

Traffic_sniffer

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab4> python .\Traffic_sniffer.py
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 78.40.125.4 Destination: 147.32.84.165 Protocol: TCP Source port: 6667 Destination port: 1181
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667
Source: 147.32.84.165 Destination: 78.40.125.4 Protocol: TCP Source port: 1181 Destination port: 6667

Total TCP packets: 20
Total UDP packets: 0
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab4>
```

As shown in the screenshot above, the Traffic sniffer script successfully executed and outputted detailed packet information (such as source and destination IP addresses, protocols, and ports), followed by the total summary of TCP and UDP packets.

Anomaly_Detector

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab4> python .\Anomaly_Detector.py
WARNING: Too many packets from 147.32.84.165
WARNING: Too many packets from 78.40.125.4
WARNING: Too many packets from 85.190.0.3
WARNING: Too many packets from 208.86.166.68
WARNING: Too many packets from 64.31.13.148
WARNING: Too many packets from 178.77.71.27
WARNING: Too many packets from 109.74.55.27

Total TCP packets: 474035
Total UDP packets: 2386
Total suspicious IPs: 7
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab4>
```

As shown in the screenshot above, the Anomaly Detector script successfully ran and flagged multiple high-frequency source IPs (such as 147.32.84.165 and 78.40.125.4) with warning messages. It also successfully calculated and displayed the final statistics, including 474,035 total TCP packets, 2,386 total UDP packets, and a total of 7 suspicious IPs.

Security analysis

In this security analysis lab of the dataset provided (botnet-capture-20110812-rbot.pcap), network traffic was inspected using Scapy. Analysis of the PCAP file revealed significant non-encrypted IPv4 traffic, including standard HTTP (TCP port 80) and DNS queries (UDP port 53). A threshold-based anomaly detector was implemented to monitor IP packet frequency. During execution, multiple high-frequency sources were flagged (e.g., 147.32.84.165 and 78.40.125.4), where packet volume exceeded 20 packets within a 5-second sliding window.

These observations suggest several potential security threats, primarily associated with botnet activity, automated scanning, or Distributed Denial of Service (DDoS) attempts. High-frequency packet bursts from specific external IP addresses indicate reconnaissance or flood attacks against target network nodes. Furthermore, the presence of unencrypted protocols allows potential adversaries to conduct passive sniffing, inspect sensitive payloads, and manipulate active connections via Man-in-the-Middle (MitM) vectors.

To reduce these security risks and protect our network, we can take several important steps. First, companies should set up smart security tools like automated Intrusion Prevention Systems. These tools can automatically slow down or block any computer address that sends too many messages too quickly. Second, we must make sure all web traffic and internal messages are fully encrypted. By using secure methods like HTTPS and encrypted domain name systems, we can keep private data safe from hackers who try to read it while it travels across the network. Third, adding machine learning technology can help the system a lot. Smart computer models can study what normal network traffic looks like, quickly adjust to new kinds of cyber threats, and spot hidden or unusual attacks much faster than humans can.

Finally, we also need to understand that passive packet capture and simple threshold-based detection methods have some clear limitations. First, fixed rate limits can sometimes cause false alarms. This happens when normal and safe network users generate a large amount of traffic, such as downloading big files or watching online videos. Second, smart attackers can easily find ways to avoid these simple rules. They might send data very slowly over a long time, or they might use many different sources to hide their communication with command-and-control servers. Therefore, relying only on basic packet counting is not enough for complete network security. To build a much stronger defense, security systems should combine simple rate checks with other helpful tools, such as studying normal user behaviors, checking the state of network connections, and looking deeply into the actual packet contents.

Github

https://github.com/bnwzc/FSCT8561-Labs-Kuangda-Zhu